Problem A. Arrangements

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

There are N children in a class, and the height of any two children is different.

Before the sports lesson, the class lined up in a single line. The teacher Pep defines the *awfulness* of the arrangement as the number of pairs of children (not necessarily standing next to each other) where the taller child is to the left of the shorter child.

We call an arrangement *extreme* if, after any two children standing next to each other swap their places exactly once, the awfulness either **always** increases or **always** decreases.

Given N, determine the number of the extreme arrangements.

Input

The first line of the input contains a single integer N ($2 \le N \le 10^4$) — the number of children.

Output

Output a single integer — the number of the extreme arrangements.

Example

standard input	standard output
42	2

Problem B. Big Qualification

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

The rules for selecting ICPC teams for the MRC from the MRC Qualification round are formulated as follows:

1. The first 25 teams advance to the next stage.
2. The best team from each university, if it has not already advanced according to rule 1 and has solved at least one problem, advances to the next stage.
3. Among the teams that did not advance according to rules 1 and 2 and representing a university with less than 4 teams have advanced according to rule 1, and occupying 2nd to 4th places among the teams from their university, 25 teams advance to the next stage (if they have solved at least one problem).

It is known that $N \geq 24$ universities participated in the qualification round, with at least four teams from each university, and any teams solved at least one problem. Determine the maximum and minimum number of teams that advance to the next stage according to these rules.

Input

The first line of input contains a single integer N — the number of universities in the qualification stage ($24 \leq N \leq 110$).

Output

Print two integers — the maximum and minimum number of teams that can advance to the next stage under the conditions described above.

Example

standard input	standard output
27	52 76

Problem C. Convex Polygon

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

As it is known from elementary geometry, the interior angles of a convex polygon must be strictly greater than 0 and strictly less than 180 degrees, and their sum must be exactly equal to $180 \cdot (n-2)$, where $n \geq 3$ is the number of vertices of the polygon.

You are given k integer angles in the range from 1 to 179 degrees inclusive. Determine whether there exists a convex polygon in which **all** angles are measured in degrees as integers, and the angles at some k consecutive vertices coincide with the given angles. If such a polygon exists, output the minimum and maximum possible number of vertices of the polygon.

Input

The first line of the input contains a single integer k ($1 \leq k \leq 1000$). Each of the following k lines contains a single integer a_i ($1 \leq a_i \leq 179$) - the value of the i-th angle in degrees.

Output

If there is no convex polygon with the required properties, output -1. Otherwise, output two integers — the maximum and the minimum number of vertices of the polygon, where the angles measured in degrees are integer and the angles at k consecutive vertices coincide with the given angles.

Examples

standard input	standard output
3 90 90 90	4 93
1 60	3 241
3 1 1 1	-1

Problem D. Difference Is Unique

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

A geometric progression G is defined as follows: $G_0 = q$, $G_{i+1} = G_i \cdot r$, where $q > 0$ and $r > 1$ are integers.

An arithmetic progression A is defined as follows: $A_0 = b$, $A_{i+1} = A_i + d$, where $a, d > 0$ are integers. d is called the *difference* of the arithmetic progression.

We call a geometric progression *nested* in an arithmetic progression A if **all** the terms of the progression G are also terms of the progression A. For example, any geometric progression with integers q and r is nested in an arithmetic progression with $a = 1$ and $d = 1$ (which includes all integers).

Given q and r, determine how many different unique integer values the difference of the arithmetic progression A, in which the geometric progression defined by q and r is nested, can take.

Input

The first line of the input contains an integer q ($1 \le q \le 10^6$). The second line of the input contains an integer r ($2 \le r \le 10^6$).

Output

Output a single integer — the number of unique integer values of the difference of the arithmetic progression, in which the geometric progression defined by q and r is nested.

Example

standard input	standard output
1 7	4

Problem E. Every Station

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

The network of railway diameters in the city of M consists of stations. Some pairs of stations are connected by bidirectional sections. One pair of stations can be connected directly by at most one section, and it is possible to travel between any two stations using one or more sections.

- If a station is connected to exactly one other station by a section (let's call it station a), it is called an *end* station. All passengers arriving at this station **must** disembark from the train, after which the train proceeds to a dead end. An empty train is sent from the dead end for boarding in the direction of station a.
- If a station is connected to exactly two other stations by a section (for example, a and b), it is called a *regular* station. The train stops at this station in both directions, and then continues its journey (i.e., it arrives from the direction of station a, stops for boarding and disembarking, and continues towards station b; in the other direction, it arrives from the direction of station b, stops for boarding and disembarking, and continues towards station a).
- If a station is connected to more than two other stations by a section, it is called a *node* station. At a node station, it is possible to switch from any section to any other section and choose the direction **arbitrarily**.

Architecturally, each station can have two types:

- If a station is an *island* station (type 1), the platform is located between the tracks, and passengers can freely disembark from a train traveling in one direction and transfer to a train traveling in the opposite direction, even if the station is not a node station.
- If a station is a *coastal* station (type 2), the tracks are located between the platforms, and in order to transfer in the opposite direction at a non-node station, a passenger must disembark from one platform, completing the paid trip, and board a train at the other platform, paying for a new trip. Note that free transfers between directions are possible at node stations **regardless** of the station's architecture.

Given a list of stations (with information about the type of each station) and the configuration of sections, determine the maximum number of stations that can be visited by paying for exactly one trip. The boarding and disembarking stations can be chosen arbitrarily. It is allowed to pass through the same stations and sections multiple times.

Input

The first line of the input contains two integers N and M — the number of stations and the number of sections, respectively. The stations are numbered consecutively from 1 to N ($2 \le N \le 10^5$, $1 \le M \le \min(2 \cdot 10^5, \frac{N(N-1)}{2})$).

The second line contains N integers a_i ($1 \le a_i \le 2$). The i-th number is 1 if the i-th station is of the island type, and 2 if it is of the coastal type.

Then follow M lines specifying the sections. Each of these lines contains two integers a and b ($1 \le a, b \le N$, $a \ne b$) - the numbers of two stations connected by a section. It is guaranteed that each pair (a, b) will appear in this list at most once, and if the pair (a, b) is present, the pair (b, a) will not be present. It is also guaranteed that it is possible to travel between any two stations using one or more sections.

Output

Output a single integer - the maximum number of stations that can be visited by paying for exactly one trip.

Examples

standard input	standard output
5 5 1 2 2 1 1 1 2 1 3 1 4 1 5 2 4	5
5 4 1 2 2 2 1 1 2 1 3 1 4 1 5	4

Note

In the second example, for instance, you can board at the second station, travel through the first station to the fifth station, transfer for free to the train in the opposite direction, travel through the first station to the third station. There, it will not be possible to change direction for free. The maximum number of stations visited with one ticket is 4.

Problem F. Find The Radius

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

This is an interactive problem

Lara Croft is caught at an unknown point with integer coordinates on a plane, which is strictly inside a circle with an integer radius R. She can rotate at any integer angle (from 0 to 359 degrees) and determine the distance to the circumference in that direction. After each rotation, Lara returns to the initial position (i.e., the angles of rotation **do not** accumulate).

You need to help her determine the radius in no more than 3 queries.

Interaction Protocol

The interaction is started by the participant's program.

To determine the distance to the boundary of the circle after rotating by n degrees, output `?` n, where n is an integer from 0 to 359, inclusively. The angle is counted from the horizontal direction each time, and the rotations do not accumulate. In response, the jury program outputs a real number — the distance to the boundary of the circle in that direction with 14 digits after the decimal point. It is guaranteed that the radius of the circle is a positive integer not exceeding 10^4, and that the point from which the queries are made has integer coordinates and is strictly inside the circle.

It is also guaranteed that the value of the radius does not change during the interaction (i.e., the interactor is not adaptive).

To output the found radius, output `!` r, where the integer r is the assumed value of the radius R. Outputting the radius is not counted as a query.

Example

standard input	standard output
5.20714203432655 5.06600306290044 13.53939201416946	? 30 ? 40 ? 270 ! 10

Note

In the example, the queries are made from a point with coordinates $(3, 4)$. The radius of the circle is 10.

Do not forget to output a newline character after each query or answer, as well as flush the input-output buffer. The `print` function in Python or the `endl` output in C++ do this automatically. You can also call the `flush` function of the programming language being used.

Otherwise, your solution may receive an "Idleness Limit Exceed" error.

Problem G. Generalized Double

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

In the compiler of the experimental language MultiDouble, ten different 64-bit real data types are implemented.

The data type is determined by the parameter k ($45 \le k \le 54$). The variable size for all those types is 64 bits (8 bytes), with the bits numbered from the least significant (0) to the most significant (63), where the rightmost bit corresponds to the value $2^0 = 1$, and the leftmost bit corresponds to 2^{63}.

The value of the variable is calculated as follows:

- Bit 63 is the sign bit: if it is 0, the number is non-negative, if it is 1, the number is negative.
- Bits 0 to $k-1$ inclusive represent the mantissa of the number — an integer m ($0 \le m \le 2^k - 1$).
- Bits k to 62 inclusive represent the exponent of the number — an integer p ($0 \le p \le 2^{63-k} - 1$).
- If $p = 2^{63-k} - 1$, then the content is not a regular number and is denoted as NaN (Not A Number).
- If $p > 0$, the absolute value of the variable is equal to $2^{p+1-2^{62-k}} \cdot (1 + m/2^k)$.
- If $p = 0$, the absolute value of the variable is equal to $2^{1-2^{62-k}} \cdot m/2^k$.

The bit numbering in the numbers p and k also goes from least significant to most significant.

The input consists of the value of k, which determines the type parameter, and an unsigned 64-bit integer x — the bit representation of the variable. Output the value of the variable in the following form: if it is equal to 0 or an **odd** integer, output the value as a single integer. If it is NaN, output the text `NaN`. In all other cases represent the value of the variable as $p \cdot 2^q$, where p is an **odd** integer and q is an integer. If $q < 0$, output the value as p`/2**`$|q|$, if q is positive, output it as p`*2**`q.

For example, the value -6 will be output as `-3*2**1`, 2023 as a single number 2023, and $22/16$ as `11/2**3`.

Input

The first line of the input contains a single integer k ($45 \le k \le 54$) — the parameter of the data type. The second line specifies the content of the variable and contains a 64-bit unsigned integer d ($0 \le d \le 2^{64} - 1$).

Output

Output the value of the variable with the given content as a 64-bit floating-point type with parameter k.

Examples

standard input	standard output
52 4608871268660281344	11/2**3
52 4656612063538315264	2023
52 13841813454723219456	-3*2**1
46 0	0
54 18446744073709551615	NaN

Problem H. Hidden Place

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

This is an interactive problem

The seats in the class 3 cars in some countries are numbered in the following way: odd seats are lower berths, even seats are upper berths, seats with numbers from 1 to 36 are located in the main section of the car, seats from 37 to 54 are side berths. The exact numbering of the seats is shown in the following illustration:



The jury program has chosen a seat p.

Your task is to guess the chosen seat by making queries of the form `?` d, where d is an integer from -54 to 54 inclusive, which should be added to the number of the current seat. At the beginning of the interaction, the number of the current seat is the same as the chosen seat (i.e., it is equal to p).

If the number of the seat obtained as a result of addition is less than 1 or greater than 54, you **immediately** lose. Otherwise, the jury program changes the current seat by adding d to it and provides information about this seat: whether it is an upper or lower berth, and whether it is a main or side berth.

You need to guess the **original** seat p in no more than 6 queries.

Interaction Protocol

The interaction starts with your program sending the first query.

Each query has the format `?` d, where $-54 \le d \le 54$ is the number you want to add to the current seat number.

If the seat is invalid, the program immediately terminates and the solution receives the verdict Wrong Answer. Otherwise, the program outputs a string of two words separated by a space. The first word is `low` if the seat is a lower berth, and `high` if the seat is an upper berth. The second word is `main` if the seat is located in the main section of the car, and `side` if the seat is a side berth.

To output the answer, use the format `!` P, where P is the assumed value of p. If the answer is correct ($P = p$) and the number of queries does not exceed 6, the solution will be accepted. Outputting the answer does not count as a query.

It is guaranteed that the value of p is determined at the beginning of the interaction and does not change during the process (i.e., the interactor is not adaptive).

Example

standard input	standard output

`? -1`
 	
`? 5`
 	
`! 33`
	
	`high main`
	
	`low side`

Note

In the example from the statement, the participant's program decided to guess the answer randomly, without being absolutely certain of the value of p. In this case, the participant's program got lucky, but it won't always be the case...

Don't forget to output a newline character after each query or answer, and also flush the input-output buffer. The `print` function in Python or the `endl` output in C++ does this automatically. You can also call the `flush` function of the programming language you are using.

Problem I. Integer Blocks

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Given an integer whose decimal representation does not contain any zeros. Find the number of ways to split the decimal representation of this number into the substrings of non-zero length that have the following property: **at most one** of the numbers represented by these substrings is **not** divisible by 3.

Since the answer can be very large, output the remainder when divided by 998 244 353.

Input

The input contains a single positive integer N, whose decimal representation does not contain any zeros ($1 \le N < 10^{100\,000}$).

Output

Output a single integer — the remainder when dividing the number of ways to split the decimal representation of N into blocks, all of which, except possibly one, are divisible by three, by 998 244 353.

Examples

standard input	standard output
142857	2
239	4

Problem J. Jewelry

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

During excavations in ancient Pompeii, jewelries with the ornaments consisting of the letters I, V, X, C, L, D, M were found, which had the following property: any two consecutive letters form a correct number in Roman numerals, but no three consecutive letters form a correct number in Roman numerals.

Archaeologists became interested in how many such ornaments exist, consisting of exactly N letters.

The table from Wikipedia shows the correct representation of Roman numerals:

Place Value	Thousands	Hundreds	Tens	Units
1	M	C	X	I
2	MM	CC	XX	II
3	MMM	CCC	XXX	III
4		CD	XL	IV
5		D	L	V
6		DC	LX	VI
7		DCC	LXX	VII
8		DCCC	LXXX	VIII
9		CM	XC	IX

Note that:

- Numbers 4, 9, 40, 90, 400, and 900 are written in reverse notation, where the first symbol is subtracted from the second (for example, for 40 (`XL`), `X` (10) is subtracted from `L` (50)). These are the **only** places where reverse notation is used.
- A number containing multiple decimal digits is constructed by appending the Roman equivalent of each digit from the highest place value to the lowest.
- If the decimal place value is 0, no digits are written in that place value in the Roman representation.
- The largest number that can be represented in the Roman numeral system is 3,999 (MMMCMXCIX).

Since the answer can be very large, output the remainder of its division by 998 244 353.

Input

The first line of input contains a single integer N ($2 \le N \le 3 \cdot 10^6$).

Output

Output a single integer — the remainder of dividing the number of ornaments of length N by 998 244 353.

Examples

standard input	standard output
2	31
3	28

Problem K. King vs Rebels

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

In the kingdom of Byteotia in the galaxy far far away, the King managed to intercept a secret channel that the rebels used for information exchange.

Each message consisted of 20 000 lines of length 9, each line is consisting of digits from 0 to 9. The King managed to find out that the orders that triggered these messages could be of two types: "strengthen the defense of a sector" and "strengthen the attack in a sector". In addition, through the Royal spies, he learned the following.

A sector is defined by a pair of integers in the form of (a, b), where $1 \le a \le b \le 9$. All numbers from $a \cdot 10^8$ to $(b+1) \cdot 10^8 - 1$ are the indices of the villages belonging to that sector.

Defensive groups are located in all villages whose numbers have the following properties:

— They are **not divisible by 3**.

— Their decimal representation contains **at least one** odd digit.

Offensive groups are located in all villages whose numbers are **prime** numbers.

After each order is issued, the rebels **randomly** and equiprobably select 20 000 villages among the villages of the specified sector, on which the corresponding groups exist (defensive groups in case of defense reinforcement or offensive groups in case of attack reinforcement). Then, the digits in the decimal representation of each selected planet's number are randomly shuffled (all possible permutations are equiprobable), and then the final message is formed.

You are given the message, intercepted by the King — 20 000 lines, each consisting of 9 digits. It is known that the message was obtained using the method described above. Your task is to decrypt the order (i.e., determine whether the defense is being strengthened or the attack is being strengthened and in which sector the order applies).

Input

The input contains 20 000 lines, each containing a nine-digit integer without leading zeros. It is guaranteed that all input data was obtained using the method specified in the condition.

Output

Output three integers: the first should be 0 if the defense is being strengthened, and 1 if the attack is being strengthened. Then, two integers $a \le b$ should be output, specifying the sector.

Examples

standard input	standard output
627353756 098260681 565527418 ... 951068257 107367722 642605294	0 5 6
169743713 544949984 974233805 ... 939373762 898391476 451989221	1 3 5

Problem L. Looking For Home Advantage

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

The Neptune Basketball Association (NBA) championship follows the following rules: 2^N teams participate in the championship. 2^{N-1} teams play in the Eastern Conference and 2^{N-1} teams play in the Western Conference. First, a regular season is played, after which the teams are ranked from 1 to 2^N, the team's place in the regular season is denoted as p_i.

The regular season table is **common** for both conferences, meaning that the distribution of conference teams in the rankings can be **arbitrary**—any combination of 2^{N-1} out of 2^N is possible. For example, it is possible that teams from different conferences alternate in the table — when the conferences are relatively balanced, or it is possible that all teams from one conference are listed first, followed by all teams from the other conference (if all teams from one conference are in crisis or are going to rebuild).

Then, a playoff is played in N rounds. The playoffs are played separately for each conference. The rules for the conference playoffs (first $N-1$ rounds) are determined as follows:

1. Before the start of the first round, the conference teams are sorted in ascending order of their rankings, forming the team order $r_{1,*}$ for the first round in the corresponding sequence. The sequences e and w are processed independently.
2. Each round is structured as follows. The team order $r_{k,*}$ for that round is taken. Let the number of teams in that round be N_k. Then, for all i from 1 to $N_k/2$, the teams $r_{k,i}$ and r_{k,N_k+1-i} play a series against each other, with the team that had a better ranking in the **regular** season (i.e., with a smaller p_j) having the **home advantage**. The winner of the series advances to the next round with the index $r_{k+1,i}$.
3. If after some round only one team advances to the next round, this team becomes the conference champion and advances to the Finals.

In other words, a team has a home advantage if it plays against a team that performed worse in the regular season, and it does not have a home advantage if it plays against a team that performed better in the regular season.

In the Finals, the winners of the Western and Eastern conferences compete for the championship title, with the team that had a lower ranking in the regular season (p_j) having the home advantage.

During an analytical show before the start of the coming NBA season, a question arose — can a team that finished in position S in the regular season win the championship with a home advantage in exactly H games?

Your task is to determine, given the values of N, S, and H, whether such a scenario is possible. If it is possible, you need to construct the distribution of conference teams in the regular season rankings and the results of all $2^N - 1$ games that realize this scenario.

Input

The first line of the input contains three integers N, S, and H ($2 \le N \le 20$, $1 \le S \le 2^N$, $0 \le H \le N$).

Output

In the first line, print "`Yes`" if the scenario described in the problem is possible, and "`No`" otherwise.

If the answer is "`Yes`", in the case of a positive answer, print the scheme of the distribution of conference teams in the regular season rankings and the results of the games in the following format:

In the first line, print 2^{N-1} integers from 1 to 2^N—the rankings of the teams representing the Eastern Conference in the regular season.

In the second line, print 2^{N-1} integers from 1 to 2^N—the rankings of the teams representing the Western Conference in the regular season. All 2^N numbers in the first two lines should be pairwise distinct.

In the third line, print a word consisting of $2^N - 1$ letters. The letter '`h`' corresponds to a victory in a series for the team that has a home advantage (i.e., with a lower ranking in the regular season), and the letter '`g`' corresponds to a victory for the team that does not have a home advantage (i.e., with a higher ranking in the regular season).

The first $2^{N-1} - 1$ letters represent the results of the series in the Eastern Conference, sorted first by the round number and then by the index with which the winner of the game advances to the next round. In other words, within round k, the game involving team $r_{k,1}$ is played first, followed by the game involving team $r_{k,2}$, and so on. The next $2^{N-1}-1$ letters represent the results of the series in the Western Conference in a similar format. The last letter represents the result of the finals.

As a result, the team that finished in position S in the regular season should win the Finals, and the number of series in which this team had a home advantage should be exactly H.

Examples

standard input	standard output
4 16 1	No
4 2 4	Yes 2 1 3 4 5 6 7 8 9 10 11 12 13 14 15 16 hhhhghhhghghhgh

Problem M. More Comfortable Travel

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

The international spaceport "KFC" has $a + b$ spots for spaceships. Out of these, a spots have a jet bridge connected to the terminal, while passengers have to take a shuttle to reach the remaining b spots.

If a passenger takes a bus from the terminal, they experience one unit of discomfort. Thus, passengers of an N-seat spaceship, which requires boarding via a bus, experience a total discomfort of N.

According to the current regulations, a spaceship is assigned a boarding spot at time s_i. Boarding is completed by time $s_i + 1$, after which the spaceship prepares for takeoff while remaining in place. The corresponding spot becomes available at time f_i.

The spaceport management has decided to change the regulations in order to reduce passenger discomfort.

Now, after the passengers have boarded, the spaceship can be moved once to an available spot before takeoff. The following conditions apply:

- The spaceship can be moved at a arbitrarily chosen by the spaceport manager time t ($s_i+1 \le t < f_i$)
- To move the spaceship, the destination spot must be available at time t.
- When the spaceship is moved, the spot it occupied during boarding becomes available starting from time t.
- Due to the movement, passengers of an N-seat spaceship experience a total discomfort of $\lfloor N \cdot c \rfloor$, where $0 \le c \le 1$ is a real constant.

Given the configuration of the spaceport, the number of different types of spots, the number of spaceships, the passenger count for each ship, and the boarding and takeoff times, determine if it is possible to adhere to the schedule. If it is possible, calculate the minimum total discomfort experienced by passengers of all specified spaceships.

Input

The first line of input contains a single integer T - the number of scenarios ($1 \le T \le 8$).

In each scenario, the first line contains three integers k, a, and b - the number of spaceships in the schedule, the number of spots equipped with a jet bridge, and the number of spots where passengers are delivered by shuttle bus, respectively ($1 \le k \le 200$, $0 \le a, b \le 30$).

The second line contains a single integer c ($0 \le c \le 1$) - the discomfort coefficient for moving a spaceship after boarding, given with at most two decimal places.

Each of the following k lines contains three integers N_i, s_i, and f_i - the number of passengers for the i-th spaceship, the start time of boarding, and the time at which the spaceship will take off, respectively ($1 \le N \le 10^5$, $1 \le s_i < f_i - 1 \le 10^9$).

Output

For each scenario, output a single integer - the minimum total discomfort experienced by passengers of all specified spaceships.

If it is not possible to adhere to the schedule, output a single number -1.

Examples

standard input	standard output
2 2 1 0 0.5 20 2 3 1 2 8 6 2 2 0.53 4 2 5 4 3 8 8 5 9 8 5 9 10 6 10 1 8 10	-1 7
2 2 1 1 0.5 10 1 2 20 2 3 2 1 1 0.5 10 1 2 20 1 3	0 10